

Project Diary — Shashank Gupta

Enrollment A2345923073, Roll 73. Group 298, B.Tech 7CSE (Evening), Amity School of Engineering and Technology. Project MAESTRO — Design and Evaluation of a Safe Multi-Agent System for Natural Language-Driven Desktop Task Automation. Guide: Dr. Rajni Sehgal Kaushik. My role in the team is architecture and the safety layer, which means I own the way an action is represented inside the system, the rules that decide whether it is allowed to run, and the job of putting the pieces the three of us build into one working program.

Week 1

Most of this week went into deciding what we were actually building, because the first version of the idea was too broad. We started with "an AI assistant for the desktop" and I kept pushing until we had a sentence that could be defended, which is how we ended up with the final title. I wrote the synopsis and fixed the objectives and the scope with the other two, and I was strict about the non-objectives as well, because a project that promises everything is easier to attack in a viva than one that says clearly what it will not do. I also drew the eight-phase PERT chart and did the technical half of the feasibility study, which meant checking that a language model of the size we need actually runs on the machine we own rather than assuming it does. It does, slowly, which is enough. At the end of the week I created the repository and the folder structure so that we were not sending each other files.

Week 2

While Seenu was collecting papers I read the language model side of the literature, because the planner is my part and everything it can do depends on how these models produce output. I read about tokenisation, context windows and decoding, and then spent most of the week on constrained decoding in particular, which is the technique that forces a model to produce output in a fixed format instead of free text. This mattered more than it sounds. If the model can only emit a plan in the shape we allow, then a whole class of nonsense never reaches the rest of the system. I wrote a small throwaway script at the end of the week that made a locally running model return a valid plan in JSON for a simple instruction, and although the plan itself was poor, the format held, which told me the approach would work.

Week 3

Seenu and Jairaj were studying the existing frameworks, so I looked at the same three systems but only at one question: how does each of them represent an action before it performs it. The answer in every case was that they do not represent it at all in any useful sense. They either generate code and run it, or they call a tool directly with whatever arguments the model produced. Nothing sits in between that a human or a program could inspect. That is the opening for our work, and it is where our design starts. I spent the rest of the week sketching the first version of what we now call the Action IR, which is a typed description of an action, its

arguments, and how its output feeds the next step, written in a form that a plain program can check without asking a model anything.

Week 4

This was the week the project got its identity. Working from the shortcomings the other two had found, I wrote the novelty statement, and the rule I settled on is the one line I now use to explain the whole project: the model proposes and deterministic code decides. The model is allowed to suggest a plan, and it is allowed to be wrong, but it is never allowed to authorise its own execution or to score its own risk. I drafted the four risk levels, from a pure read that can run on its own up to an irreversible or security-relevant action that is either typed out in full by the user or refused outright with no override. Once the levels were written I started coding them, along with the path checking that decides whether a folder is inside the permitted area, and I made both of them fail closed, meaning anything the rules do not recognise is treated as dangerous rather than safe.

Week 5

The requirement analysis was mine to write this week, so I turned the scenarios Seenu produced into numbered functional requirements, each one traceable back to an objective, and I listed the actions the system must refuse outright rather than merely warn about. Alongside that I finished the consent step and the audit log. The audit log is chained, meaning each entry carries a hash of the one before it, so if anyone edits an old line the chain no longer matches and the tampering is visible. Late in the week the pieces met for the first time and a plain English instruction went all the way through to a real result on the disk, with the preview shown first and the whole run recorded. The test suite stands at forty-eight passing tests, and three of those are tests that deliberately attack the system, including one where the planner lies about how risky its own plan is and the scorer ignores it.

Where I stand

My part of the project is ahead of where the schedule requires it to be, which is the reason I can spend the next few weeks on specification rather than on catching up. The immediate work is to freeze the Action IR, the risk levels and the trust model as version one, after which nothing new gets added to the design and anything proposed later goes into future work. After that I take the architecture chapter and the integration work through to the end of the semester.